

```

#!/usr/bin/env python3
"""Generates
data/public/pa_county_fips_crosswalk.json
(area-keyed shape)
from the shared SSOT in
pa_crosswalk_data.py – no county/FIPS/area
data is
hardcoded here anymore. See
pa_crosswalk_data.py for provenance and the
history of why a shared SSOT exists.
"""

import json
import sys
from pathlib import Path

sys.path.insert(0,
str(Path(__file__).resolve().parent))
from pa_crosswalk_data import
PA_COUNTIES_CANONICAL,
get_area_keyed_crosswalk_dict

TOTAL_PA_COUNTIES = 67

def main() -> None:
    output_dir = Path("data/public")
    output_file = output_dir /
"pa_county_fips_crosswalk.json"
    temp_file = output_dir /
".pa_county_fips_crosswalk.json.tmp"

```

```

        # pa_crosswalk_data validates itself
        (uniqueness, range, count) at import
        # time and raises AssertionError if
        anything is wrong – reaching this
        # line means the SSOT already passed
        those checks.
        if len(PA_COUNTIES_CANONICAL) !=
TOTAL_PA_COUNTIES:
            print(f"[FAIL] Expected
{TOTAL_PA_COUNTIES} counties from SSOT, got
{len(PA_COUNTIES_CANONICAL)}.")
            sys.exit(1)

        print(f"[OK] SSOT provided
{len(PA_COUNTIES_CANONICAL)} validated
counties.")
        payload =
get_area_keyed_crosswalk_dict()

        output_dir.mkdir(parents=True,
exist_ok=True)

        try:
            with open(temp_file, "w",
encoding="utf-8") as f:
                json.dump(payload, f, indent=2,
sort_keys=True)
                f.write("\n")
                temp_file.rename(output_file) #
POSIX atomic replace
                print(f"[OK] {output_file} written
successfully.")
        except Exception as exc:

```

```
        print(f"[ERROR] Atomic write  
failure: {exc}")  
        if temp_file.exists():  
            temp_file.unlink()  
        sys.exit(1)  
  
if __name__ == "__main__":  
    main()
```